

Universidad del Valle de Guatemala

Facultad de Ingeniería

Segundo Ciclo, 2026

Catedrático: Erick Marroquín



Ejercicio No. 3

Arreglos dinámicos, ArrayList, List y Excepciones

Sistema de Control de Órdenes de Servicio de un Taller

Automotriz

Nombre: Nathalia Escobar

Carné: 261136

Sección # 31

Fecha de entrega: 25/08/2026

1. Análisis

a. Tabla propiedades (atributos)

Tabla No. 1 Propiedades Taller Automotriz

Clase	Propiedad	Tipo de dato	Modificador de acceso	Descripción
OrdenServicio	numeroOrden	int	private	Almacena el número que identifica de forma única cada orden de servicio
OrdenServicio	nombrePropietario	String	private	Almacena el nombre del propietario del vehículo
OrdenServicio	placaVehiculo	String	private	Almacena la placa del vehículo asociado a la orden
OrdenServicio	descripcionServicio	String	private	Almacena la descripción del servicio a realizar
OrdenServicio	costoEstimado	double	private	Almacena el costo estimado del servicio
GestorOrdenes	listaOrdenes	List<OrdenServicio>	private	Colección dinámica (creada como ArrayList) que almacena todas las órdenes de servicio registradas actualmente
Controlador	gestorOrdenes	GestorOrdenes	private	Referencia al objeto que administra la lógica de negocio y la colección de órdenes
Controlador	vista	Vista	private	Referencia al objeto encargado de la entrada y salida de datos con el usuario

Vista	scanner	Scanner	private	Objeto utilizado para leer los datos ingresados por el usuario desde la consola
Main	NA	—	—	Clase driver; no almacena atributos, solo contiene el método main()

b. Tabla métodos

Tabla No. 2 Métodos Taller Automotriz

Clase	Método	Tipo de retorno	Modificador de acceso	Parámetros	Descripción
OrdenServicio	OrdenServicio()	N/A	public	int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado	Inicializa una orden de servicio con los datos recibidos
OrdenServicio	getNumeroOrden()	int	public	—	Retorna el número de orden
OrdenServicio	getNombrePropietario()	String	public	—	Retorna el nombre del propietario
OrdenServicio	getPlacaVehiculo()	String	public	—	Retorna la placa del vehículo
OrdenServicio	getDescripcionServicio()	String	public	—	Retorna la descripción del servicio
OrdenServicio	getCostoEstimado()	double	public	—	Retorna el costo estimado
OrdenServicio	setDescripcionServicio()	void	public	String descripcionServicio	Modifica la descripción del servicio de una orden ya registrada
OrdenServicio	setCostoEstimado()	void	public	double costoEstimado	Modifica el costo estimado de

					una orden ya registrada
OrdenServicio	toString()	String	public	—	Retorna una cadena con toda la información de la orden, usada al mostrarla
GestorOrdenes	GestorOrdenes()	N/A	public	—	Inicializa la colección dinámica de órdenes mediante un ArrayList
GestorOrdenes	registrarOrden()	void	public	int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado	Valida los datos (número no repetido, campos no vacíos, costo mayor a 0) y agrega la orden a la lista; lanza excepción si algún dato es inválido
GestorOrdenes	consultarOrdenes()	List<OrdenServicio>	public	—	Retorna la lista completa de órdenes registradas actualmente
GestorOrdenes	buscarOrden()	OrdenServicio	public	int numeroOrden	Recorre la lista y retorna la orden con el número indicado; lanza excepción si no existe
GestorOrdenes	modificarOrden()	void	public	int numeroOrden, String nuevaDescripcion, double nuevoCosto	Busca la orden por número y actualiza su descripción y costo; lanza excepción si no existe
GestorOrdenes	cancelarOrden()	void	public	int numeroOrden	Busca la orden por

					número y la elimina de la colección dinámica; lanza excepción si no existe
GestorOrdenes	consultarPorPlaca()	List<OrdenServicio>	public	String placa	Recorre la lista y retorna todas las órdenes cuya placa coincide con la indicada
GestorOrdenes	calcularValorTotal()	double	public	—	Recorre la lista y suma el costo estimado de todas las órdenes activas
GestorOrdenes	calcularCostoPromedio()	double	public	—	Calcula el promedio del costo estimado dividiendo el valor total entre la cantidad de órdenes
GestorOrdenes	obtenerOrdenMayorCosto()	OrdenServicio	public	—	Recorre la lista comparando costos y retorna la orden con el costo estimado más alto
GestorOrdenes	cantidadOrdenes()	int	public	—	Retorna el tamaño actual de la colección dinámica
GestorOrdenes	existeOrden()	boolean	private	int numeroOrden	Verifica internamente si un número de orden ya está registrado (usado por

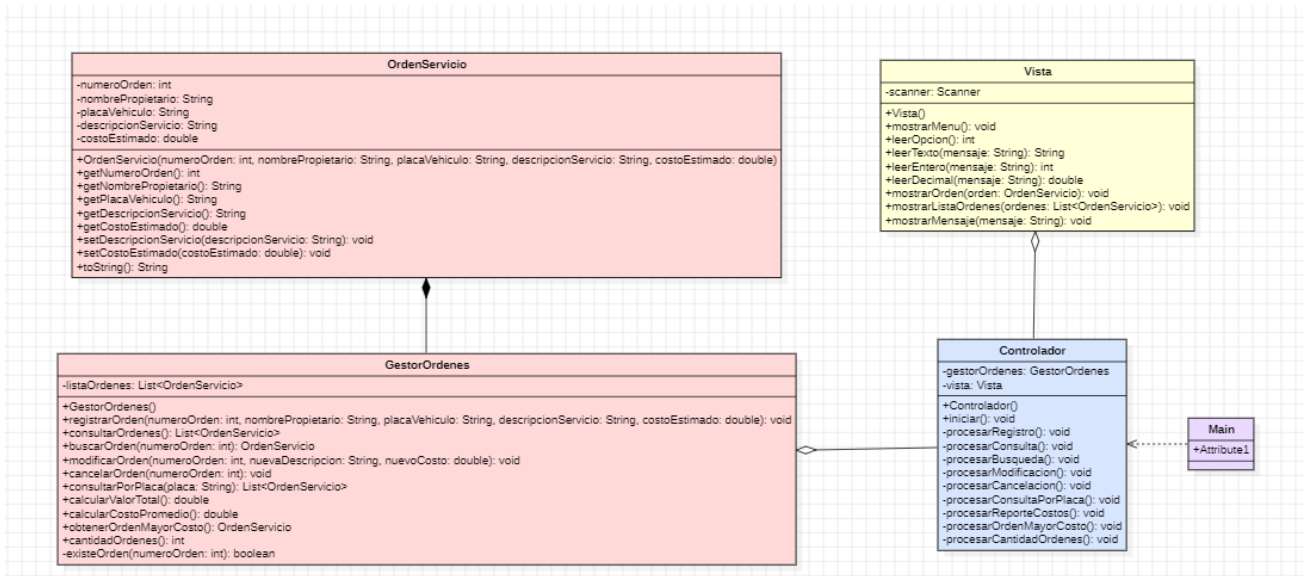
					registrarOrden)
Controlador	Controlador()	N/A	public	—	Inicializa las instancias de GestorOrdenes y Vista
Controlador	iniciar()	void	public	—	Ejecuta el ciclo del menú principal, lee la opción del usuario y la direcciona al método correspondiente
Controlador	procesarRegistro()	void	private	—	Solicita los datos de una nueva orden mediante la Vista, controla con try-catch los errores de conversión numérica (costo) y de validación, y usa finally para mostrar un mensaje de finalización del proceso
Controlador	procesarConsulta()	void	private	—	Obtiene la lista de órdenes del gestor y la muestra mediante la Vista
Controlador	procesarBusqueda()	void	private	—	Solicita el número de orden, lo busca mediante el gestor y controla con try-catch la excepción de orden no encontrada
Controlador	procesarModificacion()	void	private	—	Solicita el número de

					orden y los nuevos datos, y controla con try-catch la excepción si la orden no existe
Controlador	procesarCancelacion()	void	private	—	Solicita el número de orden a cancelar y controla con try-catch la excepción si no existe
Controlador	procesarConsultaPorPlaca()	void	private	—	Solicita la placa y muestra todas las órdenes asociadas a ella
Controlador	procesarReporteCostos()	void	private	—	Obtiene del gestor el valor total y el costo promedio, y los muestra mediante la Vista
Controlador	procesarOrdenMayorCosto()	void	private	—	Obtiene del gestor la orden de mayor costo y la muestra
Controlador	procesarCantidadOrdenes()	void	private	—	Obtiene del gestor la cantidad de órdenes registradas y la muestra
Vista	Vista()	N/A	public	—	Inicializa el objeto Scanner para leer datos de consola
Vista	mostrarMenu()	void	public	—	Muestra en consola las 10 opciones del menú principal

Vista	leerOpcion()	int	public	—	Lee la opción numérica del menú; captura la excepción si el valor ingresado no es numérico
Vista	leerTexto()	String	public	String mensaje	Muestra un mensaje y lee una cadena de texto ingresada por el usuario
Vista	leerEntero()	int	public	String mensaje	Muestra un mensaje y lee un valor entero, controlando la excepción si el dato no puede convertirse a número
Vista	leerDecimal()	double	public	String mensaje	Muestra un mensaje y lee un valor decimal, controlando la excepción si el dato no puede convertirse a número
Vista	mostrarOrden()	void	public	OrdenServicio orden	Muestra en consola toda la información de una orden específica
Vista	mostrarListaOrdenes()	void	public	List<OrdenServicio> ordenes	Recorre la lista recibida y muestra la información de cada orden
Vista	mostrarMensaje()	void	public	String mensaje	Muestra un mensaje genérico (informativo, de error o

					de confirmación)
Main	main()	void	public static	String[] args	Punto de entrada del programa; crea el Controlador e invoca iniciar()

2. Diagrama UML



3. Modelo MVC

Clase	MVC	Razón
OrdenServicio	Modelo	Almacena los datos de cada orden de servicio (número, propietario, placa, descripción y costo).
GestorOrdenes	Modelo	Mantiene la colección dinámica de órdenes y ejecuta la lógica de negocio (registrar, buscar, modificar, cancelar, calcular totales).
Vista	Vista	Se encarga exclusivamente de mostrar información al usuario y de leer los datos que este ingresa.
Controlador	Controlador	Coordina las operaciones entre el gestor de órdenes y la vista, y maneja las excepciones del sistema.
Main	Ninguno / punto de entrada	Crea los objetos necesarios e inicia el programa.